



Parul University

FACULTY OF ENGINEERING & TECHNOLOGY

BACHELOR OF TECHNOLOGY

Machine Learning (ML) Laboratory

(303105354)

6th SEMESTER

COMPUTER SCIENCE & ENGINEERING DEPARTMENT

LABORATORY MANUAL



Instructions to students

1. Every student should obtain a copy of laboratory Manual.
2. Dress Code: Students must come to the laboratory wearing.
 - i. Trousers,
 - ii. half-sleeve tops and
 - iii. Leather shoes. Half pants, loosely hanging garments and slippers are not allowed.
3. To avoid injury, the student must take the permission of the laboratory staff before handling any machine.
4. Students must ensure that their work areas are clean and dry to avoid slipping.
5. Do not eat or drink in the laboratory.
6. Do not remove anything from the computer laboratory without permission.
7. Do not touch, connect or disconnect any plug or cable without your lecturer/laboratory technician's permission.
8. All students need to perform the practical/program.

CERTIFICATE

This is to certify that

Mr. **PRIYANSHU TIWARI** with enrollment no **2303031050483** has
successfully completed his laboratory experiments in the **Machine
Learning Laboratory (303105354)** from the department of B.Tech
CSE during the academic year 2025-26



Date of Submission:

Staff In charge:

Head of Department:

INDEX

Sr. No.	Experiment Title	Page No.		Date of Performance	Date of Assessment	Marks out of 10	Sign
		From	To				
	Practical Set						
1	Dealing with Data using Numpy, Pandas, Statistics library						
2	Data Analysis & Visualization on Diwali Sales Dataset.						
3	Implementation on: a. Simple Linear Regression. b. Multi Linear Regression. c. Logistic Regression.						
4	Implement the Naïve Bayesian Classifier for a sample training Dataset stored as a CSV file. Compute the accuracy of the classifier, considering a few test data sets.						
5	Assuming a set of documents that need to be classified, use the Naïve Bayesian Classifier model to perform this task.						
6	Decision Tree-based ID3 Algorithm.						
7	Write a program to implement the K-Nearest Neighbor Algorithm to classify the iris data set.						
8	Apply EM Algorithm to Cluster a set of data stored in CSV file. Use the same data set for clustering using K-Means Algorithm.						
9	Write a program to construct a Bayesian Network considering medical data. Use this model to demonstrate the diagnosis of heart patients using standard Heart Disease dataset.						

10	Compare the various Supervised Learning Algorithm by using appropriate dataset.						
11	Compare the various Un Supervised Learning Algorithm by using appropriate dataset.						
12	Build an Artificial Neural Network by implementing the Back propagation Algorithm and test the same using appropriate datasets.						

PRACTICAL : 1

AIM: Dealing with Data using Numpy, Pandas, and Statistics Library.

CODE:

```
import numpy as np

# Create array
data = np.array([10, 20, 30, 40, 60, 70])
print("Array:", data)
print("Mean:", np.mean(data))
print("Max:", np.max(data))
print("Min:", np.min(data))

import pandas as pd

# Create DataFrame
df = pd.DataFrame({
    "Name": ["A", "B", "C"],
    "Marks": [95, 90, 88]
})

print(df)
print("\nAverage Marks:", df["Marks"].mean())

import statistics as stats

marks = [88, 91, 88]

print("Mean:", stats.mean(marks))
print("Median:", stats.median(marks))
print("Standard Deviation:", stats.stdev(marks))
```

Output:

```
... Array: [10 20 30 40 60 70]  
Mean: 38.333333333333336  
Max: 70  
Min: 10
```

	Name	Marks
0	A	95
1	B	90
2	C	88

Average Marks: 91.0

```
Mean: 89  
Median: 88  
Standard Deviation: 1.7320508075688772
```

PRACTICAL : 2

AIM: Data Analysis & Visualization on Diwali Sales Dataset.

CODE:

```
import pandas as pd
import numpy as np
import statistics as stats
import matplotlib.pyplot as plt
import seaborn as sns

#Step 1: Load and Clean Data
df = pd.read_csv('Diwali Sales Data.csv', encoding='unicode_escape')

print("Original Dataset Shape:", df.shape)
print("Columns:", df.columns.tolist())

# Drop irrelevant columns and handle nulls
df.drop(['Status', 'unnamed1'], axis=1, inplace=True, errors='ignore')
df.dropna(inplace=True)
df['Amount'] = df['Amount'].astype(int)

print("Cleaned Dataset Shape:", df.shape)

# Step 2: Exploratory Data Analysis (EDA)
amounts = df['Amount'].tolist()

print("\nBasic Stats on Sales Amount:")
print(
    f"Mean: {stats.mean(amounts):.2f}, "
    f"Median: {stats.median(amounts):.2f}, "
    f"Mode: {stats.mode(amounts)}, "
    f"Std Dev: {stats.stdev(amounts):.2f}")
```

```
# Grouped analysis
gender_sales = df.groupby('Gender')['Amount'].sum()
state_sales = df.groupby('State')['Amount'].sum().sort_values(ascending=False).head(10)
age_group_sales = df.groupby('Age Group')['Amount'].sum()
product_sales =
df.groupby('Product_Category')['Amount'].sum().sort_values(ascending=False).head(5)

print("\nTotal Sales by Gender:\n", gender_sales)
print("\nTop 10 States by Sales:\n", state_sales)
print("\nSales by Age Group:\n", age_group_sales)
print("\nTop 5 Product Categories:\n", product_sales)

# Step 3: Data Visualization
sns.set(style='whitegrid')

# 1. Bar plot: Sales by Gender
plt.figure(figsize=(8, 5))
sns.barplot(x=gender_sales.index, y=gender_sales.values, palette='viridis')
plt.title('Total Sales by Gender')
plt.ylabel('Total Amount')
plt.xlabel('Gender')
plt.show()

# 2. Bar plot: Top 10 States by Sales
plt.figure(figsize=(10, 6))
state_sales.plot(kind='bar', color='skyblue')
plt.title('Top 10 States by Sales Amount')
plt.ylabel('Total Amount')
plt.xlabel('State')
plt.xticks(rotation=45)
plt.show()
```

3. Pie chart: Sales by Age Group

```
plt.figure(figsize=(8, 8))
age_group_sales.plot(
    kind='pie',
    autopct='%1.1f%%',
    startangle=140,
    colors=sns.color_palette('pastel')
)
plt.title('Sales Distribution by Age Group')
plt.ylabel("")
plt.show()
```

4. Histogram: Distribution of Sales Amounts

```
plt.figure(figsize=(8, 5))
plt.hist(df['Amount'], bins=30, edgecolor='black')
plt.title('Distribution of Sales Amounts')
plt.xlabel('Amount')
plt.ylabel('Frequency')
plt.show()
```

5. Scatter plot: Age vs Amount with Correlation

```
ages = df['Age'].values
amounts_np = df['Amount'].values

correlation = np.corrcoef(ages, amounts_np)[0, 1]

plt.figure(figsize=(8, 5))
plt.scatter(ages, amounts_np, alpha=0.5)
plt.title(f'Age vs Sales Amount (Correlation: {correlation:.2f})')
plt.xlabel('Age')
plt.ylabel('Amount')
plt.show()
```

```
print(
    "\nAnalysis Complete! Key Insights:\n"
    "- Females dominate sales\n"
    "- Top states include UP and Maharashtra\n"
    "- Age group 26–35 spends the most\n"
    "- Clothing and electronics are top-selling categories"
)
```

Output:

Original Dataset Shape: (11251, 15)
Columns: ['User_ID', 'Cust_name', 'Product_ID', 'Gender', 'Age_Group', 'Age', 'Marital_Status', 'State', 'Zone', 'Occupation', 'Product_Category', 'Orders', 'Amount', 'Status', 'unnamed1']
Cleaned Dataset Shape: (11239, 13)

Basic Stats on Sales Amount:
Mean: 9453.61, Median: 8109.00, Mode: 7907, Std Dev: 5222.36

Total Sales by Gender:
Gender
F 74335853
M 31913276
Name: Amount, dtype: int64

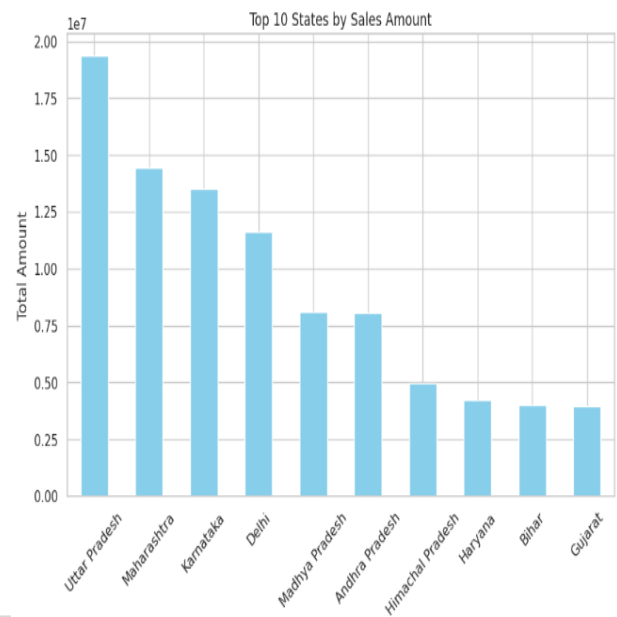
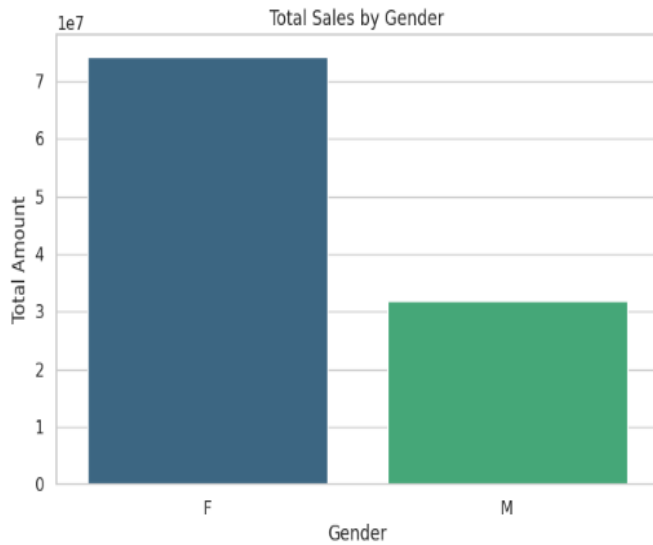
Top 10 States by Sales:
State
Uttar Pradesh 19374968
Maharashtra 14427543
Karnataka 13523540
Delhi 11603818
Madhya Pradesh 8101142
Andhra Pradesh 8037146
Himachal Pradesh 4963368
Haryana 4220175
Bihar 4022757
Gujarat 3946082
Name: Amount, dtype: int64

Sales by Age Group:
Age_Group
0-17 2699653
18-25 17240732
26-35 42613442
36-45 22144994

```
36-45    22144994
46-50    9207844
51-55    8261477
55+      4080987
Name: Amount, dtype: int64
```

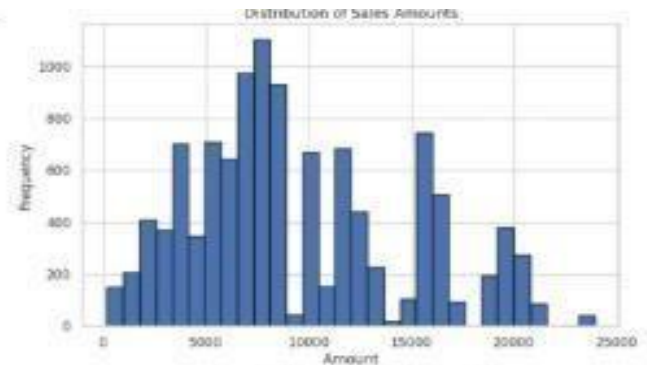
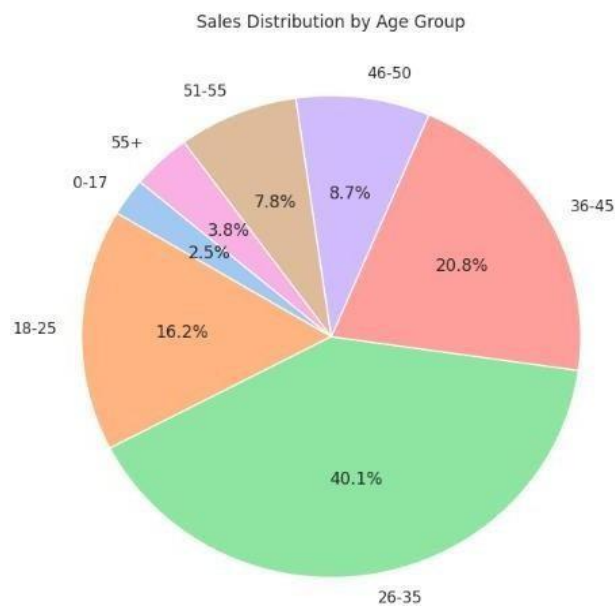
Top 5 Product Categories:
Product_Category
Food 33933883
Clothing & Apparel 16495019
Electronics & Gadgets 15643846
Footwear & Shoes 15575209
Furniture 5440051
Name: Amount, dtype: int64

```
sns.barplot(x=gender_sales.index, y=gender_sales.values, palette='viridis')
```



1e7

Top 10 States by Sales Amount



PRACTICAL : 3

AIM: Implementation on:

- a) Simple Linear Regression**
- b) Multi Linear Regression**
- c) Logistic Regression**

CODE:

Simple Linear Regression

```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error, r2_score

data = pd.read_csv("/content/student_data.csv")

X = data[['StudyHours']]
y = data['Marks']

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)

model = LinearRegression()
model.fit(X_train, y_train)

y_pred = model.predict(X_test)

print("Mean Squared Error:", mean_squared_error(y_test, y_pred))
print("R2 Score:", r2_score(y_test, y_pred))

new_hours = [[6]]
print("Predicted marks for 6 study hours:", model.predict(new_hours)[0])

plt.scatter(X_train, y_train, color='blue', label='Training Data')
plt.plot(X_test, y_pred, color='red', linewidth=2, label='Regression Line')
plt.xlabel('Study Hours')
plt.ylabel('Marks')
plt.legend()
plt.show()
```

Multi Linear Regression

```
import pandas as pd
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error, r2_score

data = pd.read_csv("/content/house_data.csv")

X = data[['Area', 'Bedrooms', 'Age']]
y = data['Price']

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)

model = LinearRegression()
model.fit(X_train, y_train)

y_pred = model.predict(X_test)

print("Mean Squared Error:", mean_squared_error(y_test, y_pred))
print("R2 Score:", r2_score(y_test, y_pred))

new_house = [[2000, 3, 5]]
print("Predicted price for the new house:", model.predict(new_house)[0])
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D
fig = plt.figure(figsize=(10, 8))
ax = fig.add_subplot(111, projection='3d')
ax.scatter(X_test['Area'], X_test['Bedrooms'], y_test, color='blue', label='Actual Data', s=50)
area_range = np.linspace(data['Area'].min(), data['Area'].max(), 30)
bedrooms_range = np.linspace(data['Bedrooms'].min(), data['Bedrooms'].max(), 30)
Area_mesh, Bedrooms_mesh = np.meshgrid(area_range, bedrooms_range)

mean_age = X_train['Age'].mean()

prediction_data_for_plane = pd.DataFrame({
    'Area': Area_mesh.ravel(),
    'Bedrooms': Bedrooms_mesh.ravel(),
    'Age': np.full_like(Area_mesh.ravel(), mean_age)
})
```

```
predicted_prices_plane = model.predict(prediction_data_for_plane)
predicted_prices_plane = predicted_prices_plane.reshape(Area_mesh.shape)

ax.plot_surface(Area_mesh, Bedrooms_mesh, predicted_prices_plane, alpha=0.5, color='red',
               label='Regression Plane')

ax.set_xlabel('Area')
ax.set_ylabel('Bedrooms')
ax.set_zlabel('Price')
ax.set_title(f'3D Scatter Plot with Regression Plane (Age fixed at {mean_age:.2f})')
from matplotlib.patches import Patch
handles, labels = ax.get_legend_handles_labels()
blue_dot = plt.Line2D([0], [0], marker='o', color='w', label='Actual Data',
                      markerfacecolor='blue', markersize=10)
red_surface = Patch(color='red', alpha=0.5, label='Regression Plane')
ax.legend(handles=[blue_dot, red_surface])

plt.show()
```

Logistic Regression

```
import numpy as np
import matplotlib.pyplot as plt

from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

X = np.array([1, 2, 3, 4, 5, 6, 7, 8, 9, 10]).reshape(-1, 1)
y = np.array([0, 0, 0, 0, 0, 1, 1, 1, 1, 1])

X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.3,
    random_state=42,
    stratify=y
)

model = LogisticRegression()
model.fit(X_train, y_train)
```

```
y_pred = model.predict(X_test)
y_prob = model.predict_proba(X_test)[:, 1]

print("Intercept:", model.intercept_[0])
print("Coefficient (Hours Studied):", model.coef_[0][0])

print("\nAccuracy:", accuracy_score(y_test, y_pred))
print("\nConfusion Matrix:\n", confusion_matrix(y_test, y_pred))

X_curve = np.linspace(0, 10, 100).reshape(-1, 1)
y_curve = model.predict_proba(X_curve)[:, 1]

plt.scatter(X_train, y_train, color='blue', label='Training Data')
plt.scatter(X_test, y_test, color='green', label='Test Data')
plt.plot(X_curve, y_curve, color='red', label='Logistic Curve')

plt.xlabel("Hours Studied")
plt.ylabel("Probability of Passing")
plt.title("Logistic Regression: Hours Studied vs Pass/Fail")
plt.legend()
plt.grid(True)
plt.show()
```

Output:

Simple Linear Regression

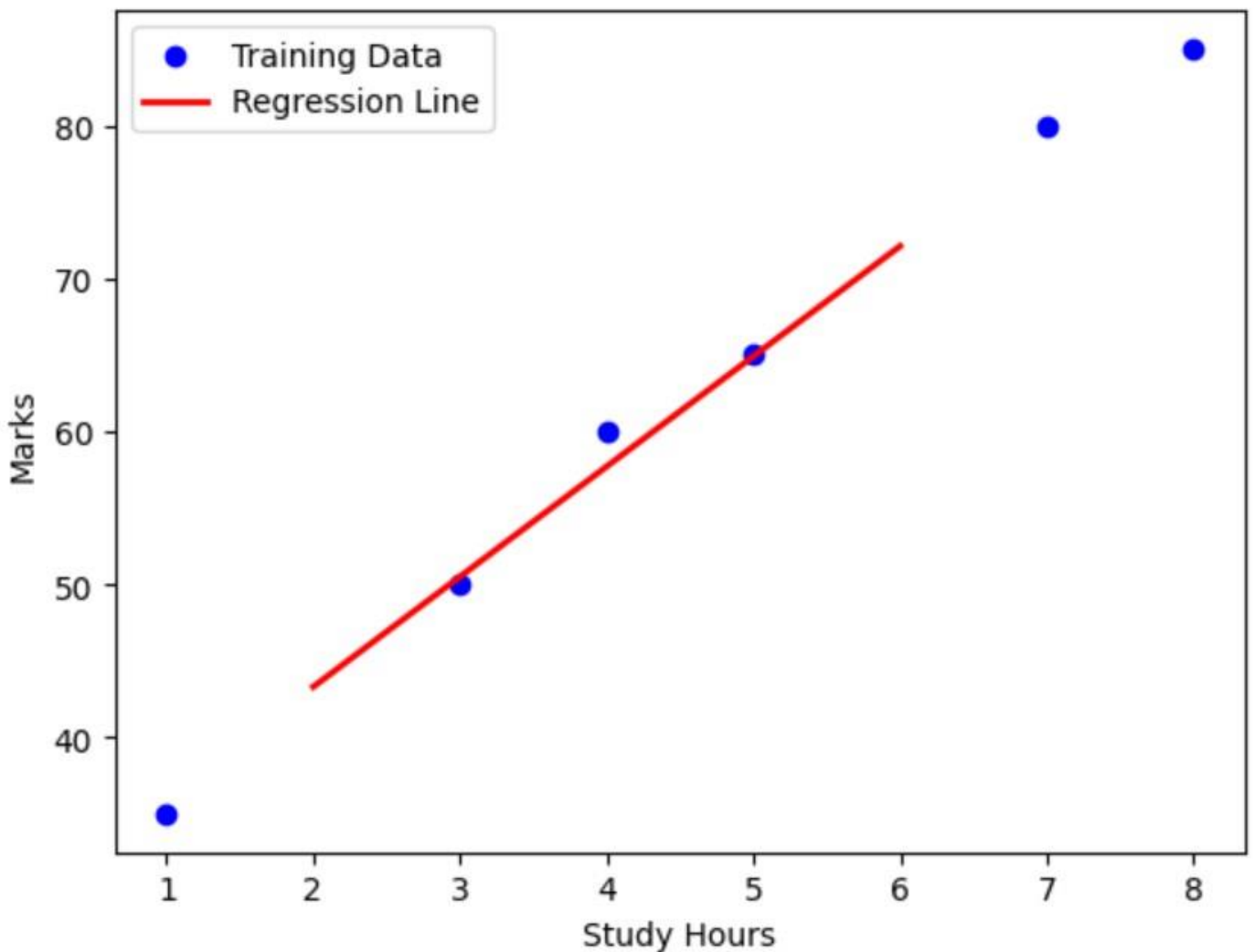
Mean Squared Error: 9.650000000000003

R2 Score: 0.9684897959183673

Predicted marks for 6 study hours: 72.1

```
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739:  
warnings.warn(  

```



Multi Linear Regression

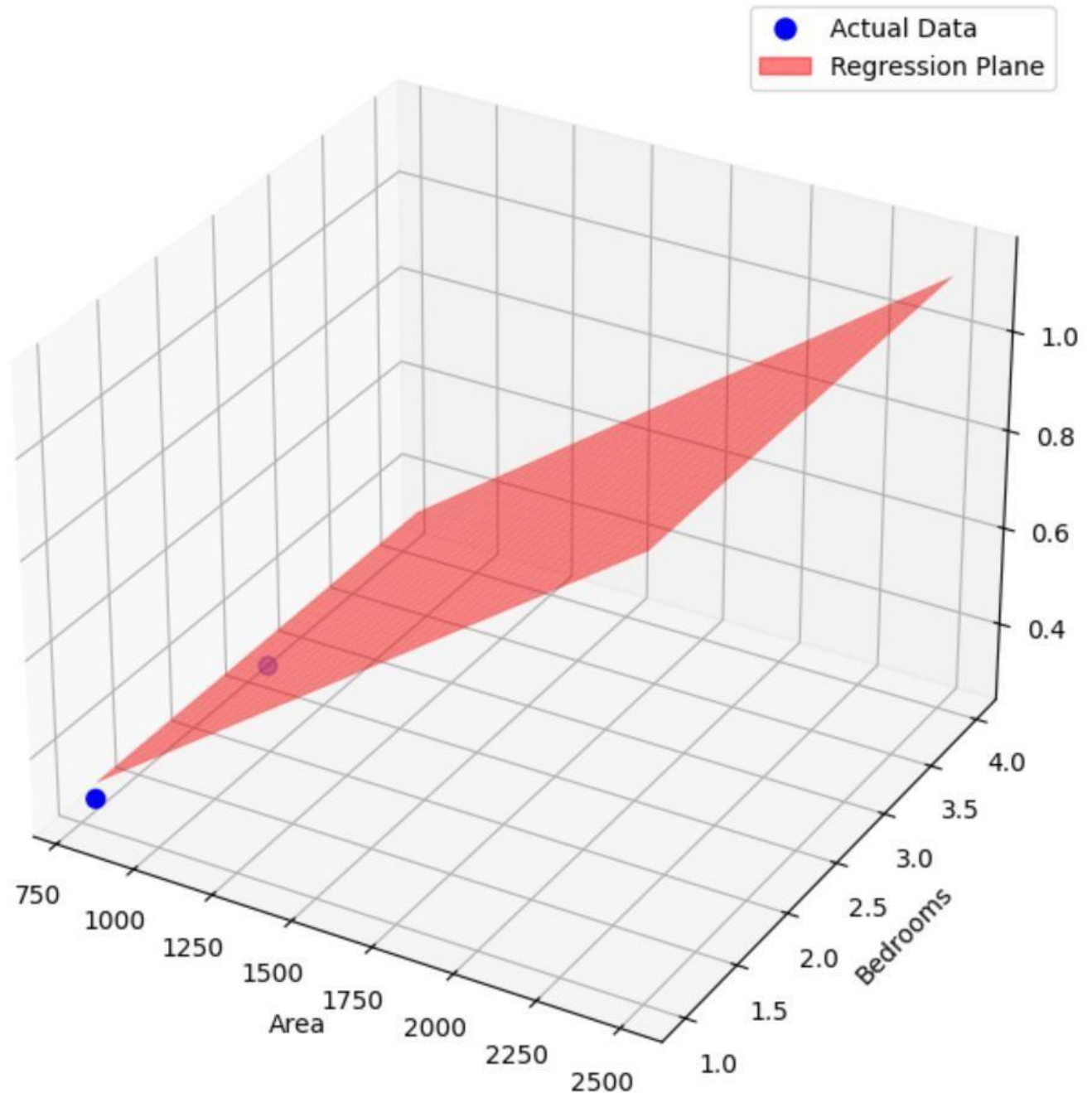
Mean Squared Error: 31895825651.57409

R2 Score: 0.9114004843011831

Predicted price for the new house: 8856828.193832602

/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning
warnings.warn(

3D Scatter Plot with Regression Plane (Age fixed at 4.00)



Logistic Regression

Intercept: -5.671829442139421
Coefficient (Hours Studied): 1.0712188802672726

Accuracy: 1.0

Confusion Matrix:

```
[[2 0]
 [0 1]]
```

